SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – Prompt Engineering Challenge

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL1, BL2)	**Assessment:**	Assessment Tool 3 – Prompt Engineering Challenge
Weightage:	40% of CIA	**Total Marks:**	100 (2 Question)
CO1 Statement:	*Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems.(BL1, BL2)*		
Student Name:	Sk Abdul Azeez	**Reg. No.:**	192472015
Section / Batch:	CSE-AI	**Date:**	01-08-2026

Instructions to Students
•Answer 2 questions. Each question carries 50 Marks. Total: 100 Marks.
• Write clear, well-structured prompts and explanations where asked.
• Use your own original examples — do not copy from classmates or the internet verbatim.
• Where a diagram or worked example is requested, label it clearly.
• Illustrate prompting techniques (zero-shot, few-shot, chain-of-thought, role/system prompting, etc.) with concrete examples, not just definitions. Time allotted: 30 mins

Code of Conduct
I Sk Abdul Azeez(192472015) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: _____

SECTION A – PROMPT ENGINEERING CHALLENGE

Q1. Zero-Shot Extraction
Write a zero-shot prompt that takes a paragraph of text and extracts all person names, dates, and locations into a clean list — without giving the model any examples.

Prompt:

You are an information extraction system. Extract all PERSON names, DATES, and LOCATIONS mentioned in the text below.

Rules:

- Only extract entities explicitly mentioned in the text — do not infer or guess.

- Normalize dates to YYYY-MM-DD format where possible; if only partial info is given (e.g. "March 2020"), keep it as written.

- List each entity once, even if it appears multiple times in the text.

- If a category has no entities, return an empty list for it.

- Output ONLY valid JSON in the exact structure below, with no extra commentary.

{

"persons": [],

"dates": [],

"locations": []

}

Text:

"""

{{PARAGRAPH}}

"""

Python code:

!pip install groq -q

import json

from groq import Groq

API_KEY = "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"

client = Groq(api_key=API_KEY)

def extract_entities(paragraph, model="llama-3.3-70b-versatile"):

prompt = f"""You are an information extraction system. Extract all PERSON names, DATES, and LOCATIONS mentioned in the text below.

Rules:

- Only extract entities explicitly mentioned in the text — do not infer or guess.

- Normalize dates to YYYY-MM-DD format where possible; if only partial info is given (e.g. "March 2020"), keep it as written.

- List each entity once, even if it appears multiple times in the text.

- If a category has no entities, return an empty list for it.

- Output ONLY valid JSON in the exact structure below, with no extra commentary.

```
{{
  "persons": [],
  "dates": [],
  "locations": []
}}
Text:
\"\"\"
{paragraph}
\"\"\"
"""
    response = client.chat.completions.create(
        model=model,
        max_tokens=1000,
        messages=[{"role": "user", "content": prompt}]  )
 raw_text = response.choices[0].message.content.strip()
  raw_text = raw_text.replace("```json", "").replace("```", "").strip()
  try:
        result = json.loads(raw_text)
    except json.JSONDecodeError:
        print(raw_text)
```

```
        return None


    return result


sample_text = """
Marie Curie was born on November 7, 1867, in Warsaw, Poland. She later moved to Paris,
France, where she conducted groundbreaking research. On December 10, 1903, she and her
husband Pierre Curie received the Nobel Prize in Physics.
"""


entities = extract_entities(sample_text)
print(json.dumps(entities, indent=2))
```

Output:


```
{
  "persons": [
    "Marie Curie",
    "Pierre Curie"
  ],
  "dates": [
    "1867-11-07",
    "1903-12-10"
  ],
  "locations": [
    "Warsaw",
    "Poland",
    "Paris",
    "France"
  ]
}
```

Q5. Structured JSON Output
Write a prompt that takes an unstructured recipe (in prose) and outputs it as valid JSON with keys:

Prompt:

You are a recipe parser. Convert the unstructured recipe text below into valid JSON.

Rules:
- Output ONLY valid JSON — no markdown fences, no extra commentary.
- Use this exact structure:

```
{
  "title": "",
  "servings": "",
  "prep_time": "",
  "cook_time": "",
  "ingredients": [
    {"item": "", "quantity": "", "unit": ""}
  ],
  "steps": [""]
}
```

- If a field is not mentioned in the text, use an empty string "" (or empty list [] for arrays).
- Split each ingredient into item, quantity, and unit separately — don't leave them combined as one string.
- Number each instruction as a separate string in the "steps" array, in the order they should be performed.
- Do not invent ingredients, quantities, or steps that aren't in the text.

Recipe:
"""
{{RECIPE_TEXT}}
"""

Python code:

```
!pip install groq -q

import json
from groq import Groq

API_KEY = "your-groq-key-here"

client = Groq(api_key=API_KEY)

def parse_recipe(recipe_text, model="llama-3.3-70b-versatile"):
```

```
    prompt = f"""You are a recipe parser. Convert the unstructured recipe text below into valid JSON.

Rules:
- Output ONLY valid JSON — no markdown fences, no extra commentary.
- Use this exact structure:

{{
  "title": "",
  "servings": "",
  "prep_time": "",
  "cook_time": "",
  "ingredients": [
    {{"item": "", "quantity": "", "unit": ""}}
  ],
  "steps": [""]
}}

- If a field is not mentioned in the text, use an empty string "" (or empty list [] for arrays).
- Split each ingredient into item, quantity, and unit separately — don't leave them combined as one string.
- Number each instruction as a separate string in the "steps" array, in the order they should be performed.
- Do not invent ingredients, quantities, or steps that aren't in the text.

Recipe:
\"\"\"
{recipe_text}
\"\"\"
"""

    response = client.chat.completions.create(
        model=model,
        max_tokens=1000,
        messages=[{"role": "user", "content": prompt}]
    )

    raw_text = response.choices[0].message.content.strip()
    raw_text = raw_text.replace("```json", "").replace("```", "").strip()

    try:
        result = json.loads(raw_text)
    except json.JSONDecodeError:
        print(raw_text)
        return None
```

```
    return result

sample_recipe = """
Classic Pancakes

This easy pancake recipe serves 4 people and takes about 10 minutes to prepare and 15 minutes to cook.
You'll need 2 cups of flour, 2 tablespoons of sugar, 2 teaspoons of baking powder, a pinch of salt, 2 eggs, 1.5 cups of milk, and 3 tablespoons of melted butter.

First, whisk together the flour, sugar, baking powder, and salt in a large bowl.
In a separate bowl, beat the eggs, then mix in the milk and melted butter.
Pour the wet ingredients into the dry ingredients and stir until just combined.
Heat a lightly greased griddle over medium heat.
Pour 1/4 cup of batter for each pancake and cook until bubbles form on the surface, about 2-3 minutes.
Flip and cook the other side until golden brown, about 1-2 minutes.
Serve warm with syrup.
"""

recipe_json = parse_recipe(sample_recipe)
print(json.dumps(recipe_json, indent=2))
```

Output:


```
{
  "title": "Classic Pancakes",
  "servings": "4",
  "prep_time": "10 minutes",
  "cook_time": "15 minutes",
  "ingredients": [
    {"item": "flour", "quantity": "2", "unit": "cups"},
    {"item": "sugar", "quantity": "2", "unit": "tablespoons"},
    {"item": "baking powder", "quantity": "2", "unit": "teaspoons"},
    {"item": "salt", "quantity": "1", "unit": "pinch"},
    {"item": "eggs", "quantity": "2", "unit": ""},
    {"item": "milk", "quantity": "1.5", "unit": "cups"},
    {"item": "melted butter", "quantity": "3", "unit": "tablespoons"}
  ],
  "steps": [
    "Whisk together the flour, sugar, baking powder, and salt in a large bowl.",
    "In a separate bowl, beat the eggs, then mix in the milk and melted butter.",
    "Pour the wet ingredients into the dry ingredients and stir until just combined.",
    "Heat a lightly greased griddle over medium heat.",
    "Pour 1/4 cup of batter for each pancake and cook until bubbles form on the surface, about 2-3 minutes.",
```

```
    "Flip and cook the other side until golden brown, about 1-2 minutes.",
    "Serve warm with syrup."
  ]
}
```

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technique Selection	30%	Correctly applies zeroshot/oneshot/few-shot & CoT technique appropriate to each task	Applies techniques with minor mismatch to task	Limited understanding of which technique to use	Techniques applied incorrectly or not evident
Output Effectiveness	25%	Prompts produce accurate, taskappropriate outputs across all assigned tasks	Prompts work well for most tasks	Prompts work for only a few tasks	Prompts are largely ineffective
Prompt Craftsmanship	20%	Prompts are clear, specific, and reusable	Prompts are mostly clear	Prompts are vague or inconsistent	Prompts are poorly constructed
Prompt-Output Documentation	15%	Welldocumented prompt-output pairs with clear rationale	Documented with minor gaps	Minimal documentation of prompts/outputs	No rationale provided for prompt choices
Design Rationale	10%	Clearly explains design choices and iterations made	Explains most design choices	Limited explanation of process	No explanation of process provided

Marks Evaluation:

Question No.	Technique Selection (30%)	Output Effectiveness(25%)	Prompt Craftsmanship (20%)	Prompt-Output Documentation (15%)	Design Rationale(10%)	100
Q1						
Q2						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____

Date: _____	Date: _____	Date: _____